



STARVNT CUSTOMER APP

Product Direction + UX/UI North Star + Implementation Brief

NORTH STAR

**The Customer App is not a dashboard for viewing an event.
It is a control center for planning, understanding and executing the event.**

Easy like Swiggy | Visual like Netflix | Simple like Apple | Premium like a luxury event planner | Intelligent through Aura+

Prepared for: Keshab - Technical Lead / Core Commerce + Customer Experience Engineering



01 | Executive Product Direction

StarVnt Customer App should feel like a Personal Event Control Center rather than a generic event-management dashboard. The customer should understand the current state of the event, the next priority, planning progress, budget, vendors, payments, tasks and human support without learning StarVnt's internal operational complexity.

The emotional outcome

"Everything is under control."

The product combines Event Planning + Discovery + Vendor Management + Tasks + Budget + Payments + Timeline + Communication + Aura+ into one coherent customer experience. The backend may be complex; the frontend must remain clear.

Reference images in this brief are visual and UX inspiration only. They are not to be copied screen-for-screen. Extract their common UX DNA - event snapshot, countdown, progress, tasks, budget, guests, vendors, timeline, clear navigation, visual discovery and premium mobile usability - then translate it into StarVnt.

02 | Product Principles

At every point, the app should answer five questions:

- Where am I? - current event / current section.
- What has happened? - completed tasks, confirmed vendors, completed payments, approvals.
- What is happening now? - current planning status.
- What happens next? - upcoming milestone / next event / next task.
- What do I need to do? - one clear priority action.
- Can StarVnt help me? - Event Manager + Aura+.

Complexity belongs behind the interface.

The customer should never need to understand internal workflows, database models, team structures or operational complexity.

Luxury principle: Luxury does not mean more decoration. Luxury means control, clarity and confidence.

03 | Navigation + Home Command Center

Primary navigation: Home → Explore → Aura+ → Bookings → Profile

Aura+ should have a central / hero position because it represents the StarVnt intelligence layer. The exact navigation implementation can evolve if usability improves without changing the core product hierarchy.

Home must answer: “What is happening with my event right now?”

Layer	What the customer sees
1. Event Identity	Subho & Priya Wedding / Wedding
2. Countdown	47 Days to Go
3. Planning Progress	68% Planning Complete
4. Priority Action	Approve Wedding Decor - Due Tomorrow - View
5. Financial Snapshot	₹10L Budget - ₹6.8L Committed - ₹3.2L Available
6. Quick Access	Vendors - Tasks - Budget - Payments
7. Human Support	Your StarVnt Event Manager - Call / WhatsApp / Message
8. AI	Ask Aura+ - “What should I do next?”

Golden rule

Home should be a priority surface, not a data dump. Everything deeper can be one tap away.

04 | Core Customer Modules

1. My Event - Complete event overview, dates, locations, event status and event context.
2. Tasks / Checklist - Personalized tasks, deadlines, reminders, pending actions and completed tasks.
3. Vendors - Selected vendors, services, portfolios, booking status, responsibilities and relevant communication.
4. Budget - Total budget, allocations, committed amount, paid amount, pending amount, remaining budget and budget health.
5. Payments / Wallet - Upcoming installments, payment history, invoices, receipts, outstanding balances and supported payment actions.
6. Timeline / Schedule - Event-day timeline, timings, responsibilities, next activity and eventually live event-day status.
7. Guests - Guest list, RSVP, invitations, attendance tracking and guest information.
8. Documents / Files - Proposals, agreements, invoices, receipts and important event documents linked to their context.
9. Messages - Customer ↔ StarVnt communication and relevant vendor communication where applicable.
10. Aura+ - Event-aware AI planning, recommendations, next actions and contextual decision support.

05 | Budget → Plan → Execution

Critical product decision

Budget is planning intelligence. Payments are transaction management. They are connected, but conceptually separate.

If a customer enters a ₹10,00,000 wedding budget, the platform should not simply display the number. The budget becomes a planning foundation for allocation, requirements, vendor discovery, bookings, tasks, payments and timeline.

Core flow

Budget → Plan → Allocation → Requirements → Vendors → Bookings → Tasks → Payments → Timeline → Execution

Example category	Illustrative allocation
Venue	₹2,50,000
Catering	₹2,00,000
Decor	₹1,25,000
Photography & Videography	₹75,000
Entertainment / DJ	₹50,000
Makeup & Styling	₹40,000
Invitations	₹20,000
Transportation	₹25,000
Guest Management	₹25,000
Miscellaneous / Contingency	₹90,000

These numbers are illustrative. Actual allocations should be dynamic based on event type, guest count, location, date, requirements, preferences and priorities.

Budget visibility

Total Event Budget · Recommended / Planned Allocation · Allocated Amount · Committed / Booked Amount · Amount Paid · Pending Amount · Remaining / Available Budget · Category-wise spending · Vendor-wise spending · Upcoming due dates · Invoices / receipts

Budget impact

A new booking or service should update the customer's financial overview immediately. Example: ₹10L total budget; ₹6.8L committed; ₹4.25L paid; ₹2.55L pending; ₹3.2L available. Adding a ₹50K service should show the new impact and warn if the planned budget is exceeded.

06 | Vendors + Tasks + Timeline

Vendor status example: ✓ Venue - Confirmed; ✓ Decor - Confirmed; ✓ Photography - Confirmed; ● Makeup - Selecting; ● DJ - Pending; ○ Catering - Not Started.

Vendor detail can eventually include name, category, service, portfolio, package, price, availability, booking status, payment status, proposal, documents, communication, shortlist state, approval state and booking history.

Tasks exist to reduce customer mental load, not merely to display a checklist. Each task should have title, description, priority, due date, status, responsible party, linked vendor/service and CTA.

Timeline

Use a clear chronological structure such as 10:15 AM Bride Gets Dressed → 10:45 AM First Look → 11:15 AM Brides & Bridesmaids Photos → 11:45 AM Groom & Groomsmen Photos → 12:15 PM Couple Portraits → 2:15 PM Guests Arrive → 3:00 PM Ceremony Begins. Long term, Timeline should evolve into a live event-day control layer.

Potential future connections: vendors, Event Manager, tasks, locations, notifications, real-time updates, event-day status, production milestones, artist schedules, photography schedules and guest movement.

07 | Aura+ - Event-Aware Intelligence

Aura+ is not “ChatGPT inside an event app.”

Aura+ should understand the customer's actual event context and help the customer decide and act.

Context can include event type, date, location, guest count, budget, budget allocation, selected vendors, pending vendors, tasks, planning progress, payment status, timeline, preferences and bookings.

Example

Customer: “What should I do next?”

Aura+: “You have 32 days left. Photography is confirmed, but catering and DJ are still pending. I recommend finalizing catering this week and shortlisting the DJ within the next 3 days.”

Budget intelligence

If a customer wants to add a ₹50K live artist, Aura+ should understand the current budget context and explain the new available budget or recommend reallocation where appropriate.

Action-oriented future

“Show me photographers under ₹75K.” → filter vendors. “Shortlist this photographer.” → shortlist.

“Create my Sangeet checklist.” → create tasks. “What payments are due this month?” → show relevant payments. “Book the decor package.” → start a supported booking flow.

All actions must remain inside normal authentication, authorization, business rules, confirmation and auditability. Aura+ must not bypass core transactional controls.

08 | Bookings + Profile + Trust

Bookings should make the customer's commercial lifecycle visible: Discovery → Request → Proposal → Approval → Booking → Payment → Execution → Completion.

Booking detail can contain service, vendor, package, price, booking status, payment status, event, date, documents, invoice, receipt, relevant communication and execution status.

Profile should focus on account-level information: personal details, contact information, preferences, notifications, documents, privacy, security, support and settings. Event-specific information should live primarily inside the event experience.

Trust is a product feature. The interface should make it clear who is responsible, what has been approved, what is pending, what has been paid and what happens next.

Human + AI

Aura+ provides intelligence; the StarVnt Event Manager provides human judgement, communication, coordination, execution and problem solving.

09 | Visual System + Product Quality

Primary visual identity: Obsidian Black + Platinum White + Electric Aura Blue. Premium accent: Champagne Gold used selectively for wedding / premium moments.

Visual character: premium, cinematic, modern, confident, minimal, editorial, high-end, technology-driven, human and trustworthy.

Avoid generic SaaS, generic CRM, spreadsheet-like density, unnecessary gradients, excessive cards, noisy hierarchy and decorative motion that does not serve a purpose.

Micro-interactions: subtle page transitions, progress animation, countdown updates, task completion, vendor status transitions, payment success, bottom sheets, modal transitions and Aura+ response states.

Mobile-first: thumb reach, one-handed interactions, readable typography, strong touch targets, bottom navigation, safe areas, keyboard behaviour, loading performance, network conditions and real Android/iOS screen sizes.

Application states: empty, loading, pending, confirmed, failed, overdue, completed, no-action-required, first-time user, network error, permission error.

Notifications: action-required, payment due, planning reminder, vendor confirmation and Aura+ priority alerts. Avoid notification spam.

10 | Next.js Implementation Direction

Frontend requirement

Implement the customer-facing web experience using Next.js, integrated cleanly with the approved StarVnt Core APIs and architecture.

The UI should not be implemented as a standalone visual prototype disconnected from the Core Platform. The Next.js layer should consume controlled APIs and render customer-authorized event data.

Recommended application structure

Next.js App Router → route groups → feature modules → shared UI components → data/query layer → API client → authentication/session → state management → analytics / error handling.

Suggested route map

/home · /explore · /aura · /bookings · /profile · /event/[eventId] · /event/[eventId]/tasks ·
/event/[eventId]/vendors · /event/[eventId]/vendors/[vendorId] · /event/[eventId]/budget ·
/event/[eventId]/payments · /event/[eventId]/timeline · /event/[eventId]/guests ·
/event/[eventId]/documents · /event/[eventId]/messages

Implementation principles

- Use reusable typed components rather than one giant dashboard component.
- Keep server/client boundaries intentional; do not make the entire app client-rendered by default.
- Centralize API access and error handling.
- Use typed schemas for API responses and user actions where the backend contract permits.
- Keep customer authorization enforced by the backend; the frontend is not the security boundary.
- Design loading, empty, error and permission states as first-class components.
- Optimize images and media; use appropriate caching, pagination and lazy loading.
- Avoid hard-coding event or budget values in production UI.
- Build a consistent design system so Home, Explore, Bookings and event sub-pages feel like one product.
- Flag API/data-model gaps before creating UI workarounds.

Architecture alignment

For each screen: UI requirement → user action → API requirement → database data → permission → business logic → integration → state management → error handling.

11 | Customer App Screen Map

Home - Event identity, countdown, planning progress, next action, budget snapshot, vendor/task status, manager and Aura+.

Explore - Categories, inspiration, services, vendor discovery, shortlist and request / booking entry.

Aura+ - Event-aware conversation, recommendations, next actions, budget intelligence and future supported actions.

Bookings - Active, upcoming and past bookings; booking details; proposal / payment context.

Profile - Account settings, preferences, notifications, documents, support.

Event Overview - Event details, dates, location, guests, planning status.

Tasks - Priority tasks, deadlines, status, linked services and actions.

Vendors - Selected vendors, discovery, shortlist, status, communication.

Vendor Detail - Portfolio, package, price, availability, proposal, booking state.

Budget - Total, allocation, committed, paid, pending, remaining, category/vendor spend, health.

Payments - Upcoming, paid, pending, due date, invoice, receipt, Pay Now.

Timeline - Chronological schedule, next activity, event-day state.

Guests - Guest list, RSVP, invitations, attendance.

Documents - Proposals, agreements, invoices, receipts, event files.

Messages - Customer ↔ StarVnt and contextual vendor communication.

Event Manager - Contact card, call, WhatsApp, message, support context.

12 | Screen-wise Data / API Dependency Map

Before full UI build, identify what each screen needs from the platform so frontend decisions do not outrun backend contracts.

Screen	Data required	Typical API / source	Primary actions
Home	Event, countdown, progress, tasks, vendors, Customer pay Events, Personalized aggregation	Core, Customer pay Events, Personalized aggregation	View, aggregation, next action, open modules, contact
Explore	Categories, offerings, vendors, location	Marketplace / Offering / Search APIs	Filter, view, compare, shortlist, request
Vendors	Selected vendors, services, booking status	Partner / Offering / Booking APIs	View, communicate, approve where allowed
Budget	Budget, allocations, commitments, paid	Booking, Commerce financial read APIs	Adjust allocation, review impact
Payments	Payment records, due dates, invoices, receipts	Payment / Booking APIs	View, pay, download
Tasks	Tasks, deadlines, priorities, dependencies	Task / Event planning APIs	Complete, defer, view linked context
Timeline	Activities, dates, time, responsible party, status	Event timeline / operations APIs	View, acknowledge, later update where permitted
Aura+	Event context + authorized commerce	Commerce orchestration + Core APIs	Ask, recommend, initiate supported actions

Important: exact endpoint names depend on the approved Core API contract. Do not invent client-side sources of truth when the Core owns the data.

13 | Implementation Phases

Phase 1 - Customer Foundation

- Authentication
- Customer profile
- Event access / creation
- Dashboard
- Event details
- Countdown
- Planning progress
- Tasks
- Vendor status
- Basic bookings
- Basic payment visibility
- Event Manager information

Goal: Customer can understand the event and current responsibilities.

Phase 2 - Planning Experience

- Explore
- Vendor discovery
- Vendor detail
- Shortlisting
- Proposals
- Approvals
- Budget planning
- Event timeline
- Planning recommendations

Goal: Customer can actively plan the event inside StarVnt.

Phase 3 - Communication

- Event Manager communication
- Notifications
- Vendor communication where applicable
- Documents
- Proposals
- Invoices
- Receipts

Goal: Customer can communicate and manage event documentation from one place.



Phase 4 - Aura+

- Event context
- Event-aware AI
- Planning recommendations
- Next-action recommendations
- Task recommendations
- Budget intelligence
- Vendor recommendations
- Natural-language event queries

Goal: Aura+ becomes the intelligence layer.

Phase 5 - Deeper Automation

- Aura+ actions
- Intelligent prioritization
- Dynamic planning
- Automated task generation
- Budget optimization
- Event-day intelligence
- Real-time updates
- Deeper vendor intelligence
- Automated workflows

Goal: StarVnt evolves into an intelligent event operating system.

14 | UX / Technical Guardrails

Do not silently change core product logic. If a better UX or technical approach is identified, surface it before implementation.

Process

Identify → Explain → Recommend → Align → Implement

Guardrails

- Customer App is not the Admin Panel.
- Customer financial data must exclude internal StarVnt margins, commissions, vendor settlements, profit and confidential cost structures.
- Aura+ cannot bypass authentication, authorization, business rules, confirmations or audit requirements.
- Frontend cannot be treated as the security boundary.
- Reference images are inspiration, not templates.
- Do not optimize for number of screens; optimize for customer outcomes.
- Do not build beautiful UI that the backend cannot support.
- Do not let current backend limitations destroy the long-term product vision; flag gaps and design contracts.

15 | Required Pre-Coding Deliverables

Before the complete UI build starts, please return the following six items:

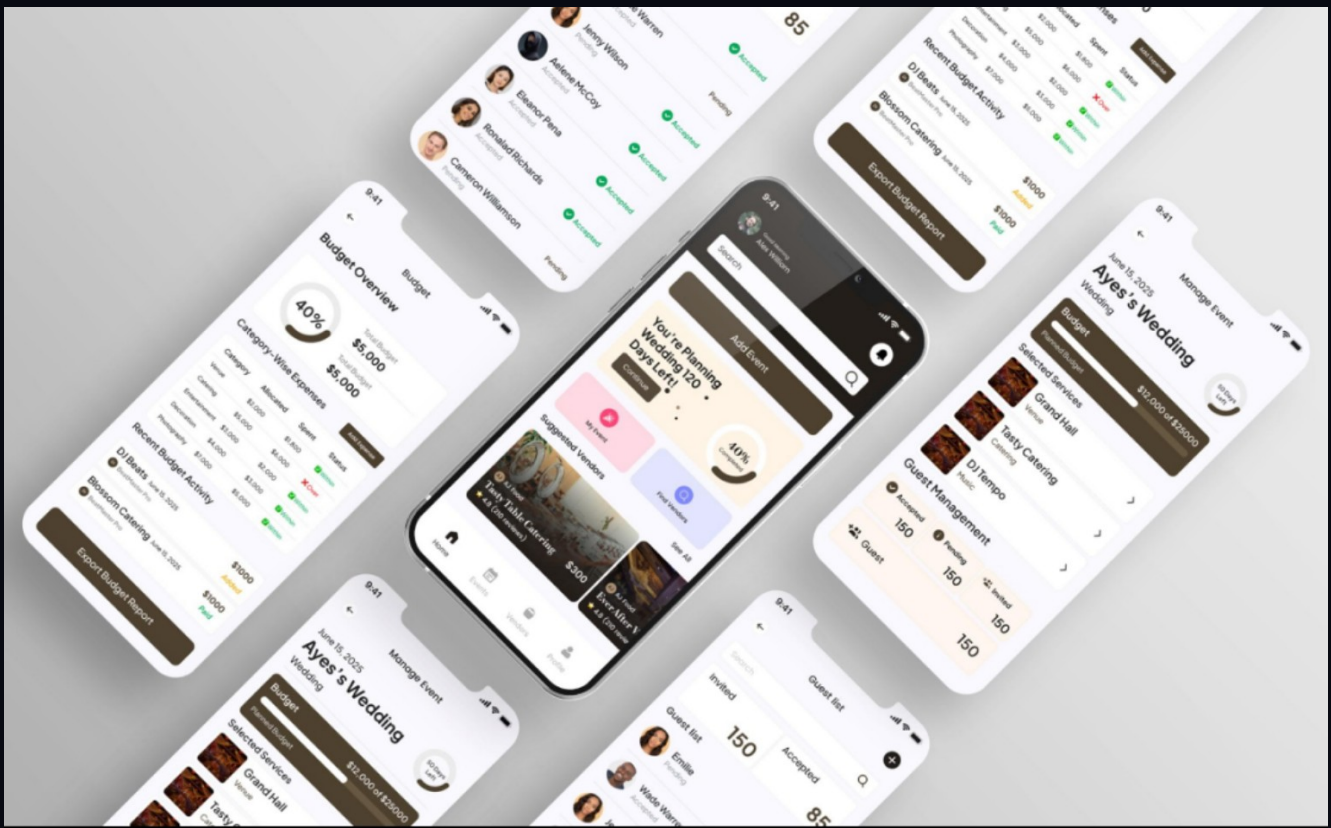
1. Customer App Screen Map - Every major screen, sub-screen and relationship.
2. Navigation & User Flow - Where the user starts, what they see, action, result and next step.
3. Screen-wise Data/API Dependency Map - Data required, API/source, actions, state changes and permissions.
4. MVP vs Future Feature Separation - MVP, V1/Post-MVP and future intelligence/automation.
5. UX / Technical Risks + Recommendations - Problem → Impact → Recommended Solution.
6. Proposed Implementation Approach - Next.js structure, component plan, data fetching, state, API integration, testing and delivery sequence.

Approval gate

Review → Align → Approve → UI build. Do not jump straight into full UI implementation before the customer journey and information architecture are validated.

16.1 | Visual Reference 1

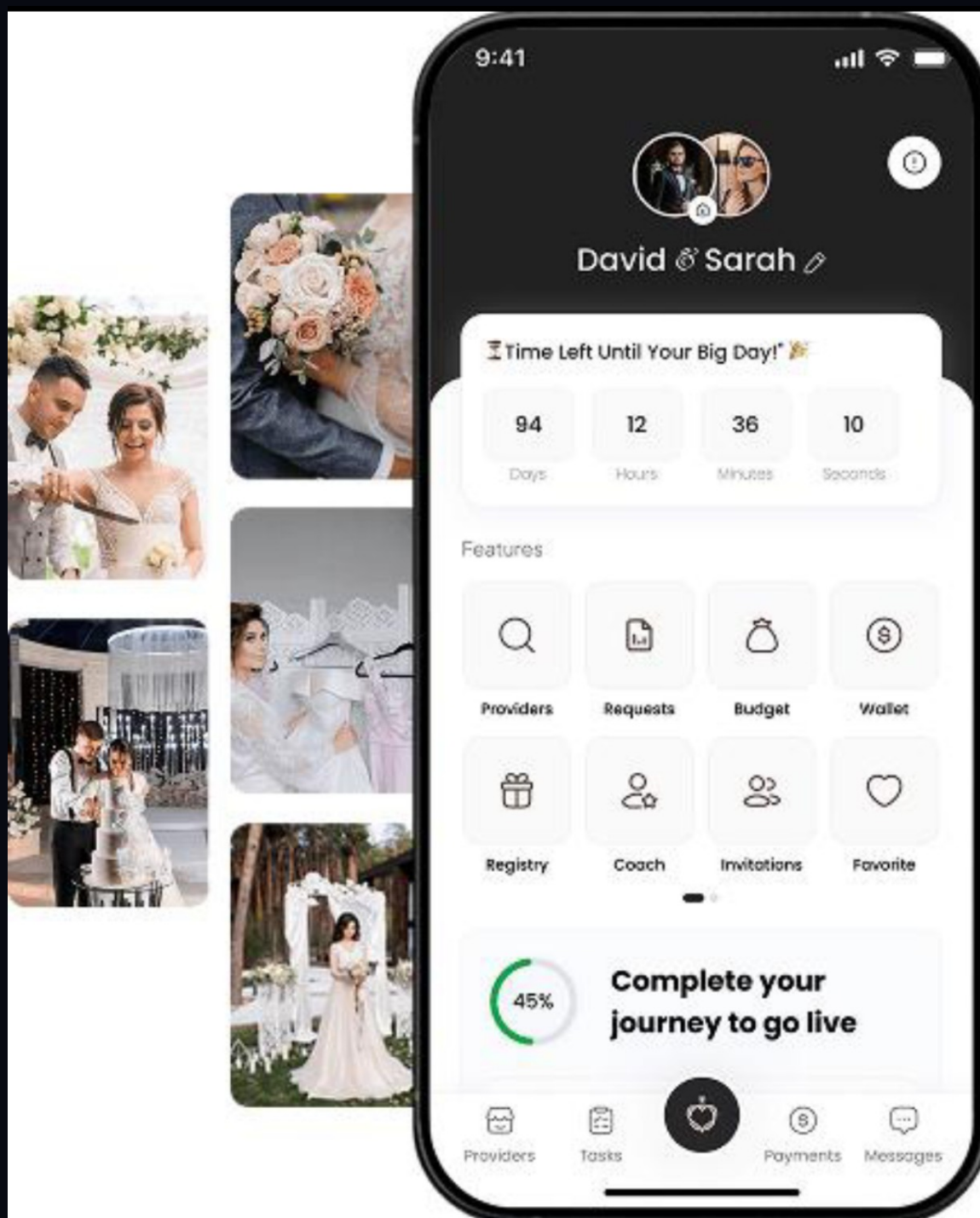
Reference 01 - Multi-screen event planning / budget / provider experience



Use for multi-surface inspiration: budget overview, service/provider selection, event management and premium app composition. Extract the hierarchy - do not clone layouts.

16.2 | Visual Reference 2

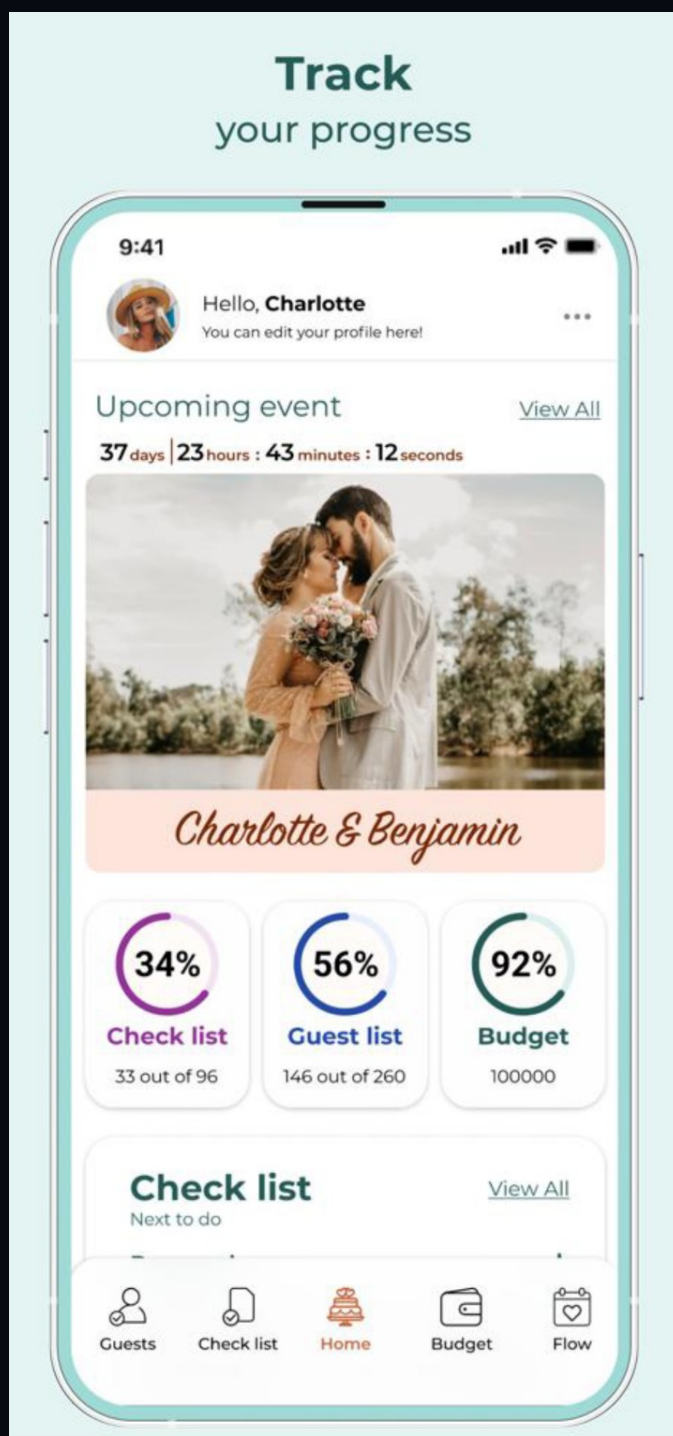
Reference 02 - Personal event home / countdown / quick-access command center



Use for personal event home: countdown, quick access, progress, providers, requests, budget, wallet, guests and favorites. Translate into the StarVnt Home information hierarchy.

16.3 | Visual Reference 3

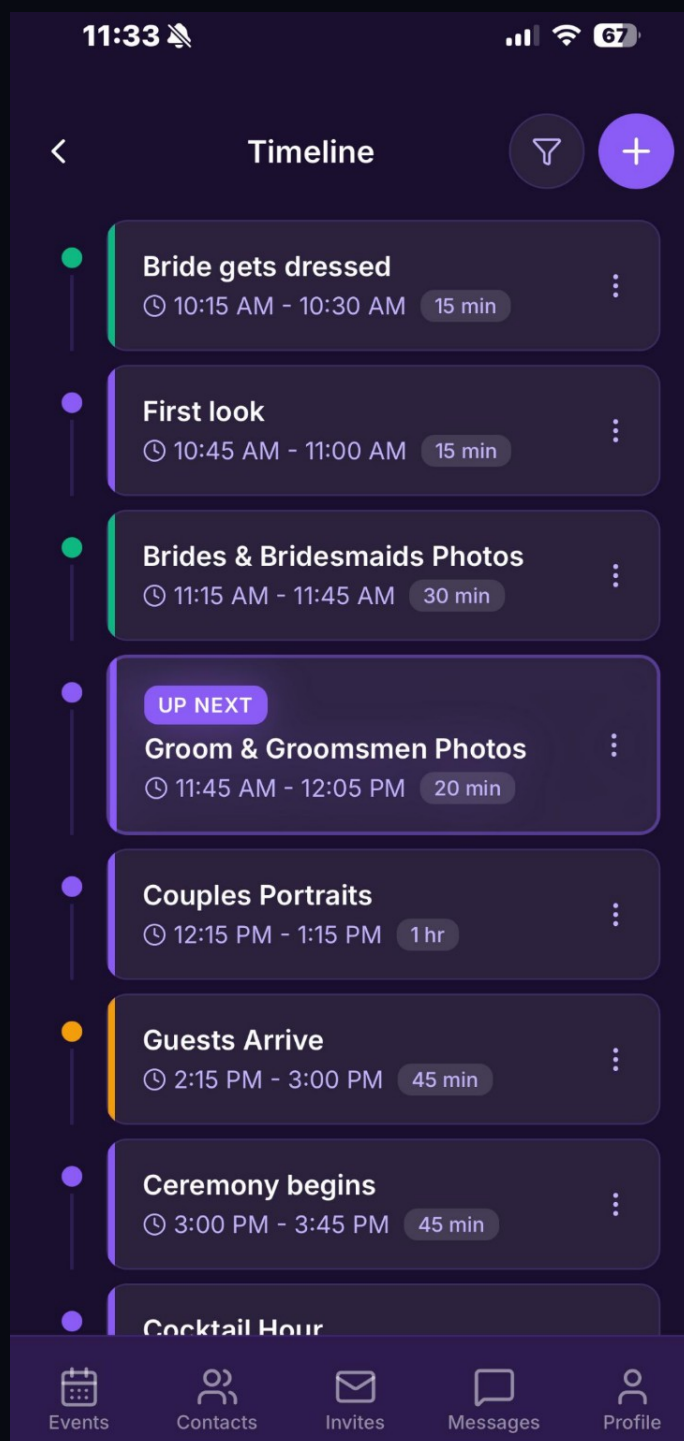
Reference 03 - Progress-first event dashboard / budget / checklist / guest tracking



Use for progress-first planning: checklist, guest progress and budget visibility. StarVnt should integrate these concepts with next-action prioritization and Aura+ context.

16.4 | Visual Reference 4

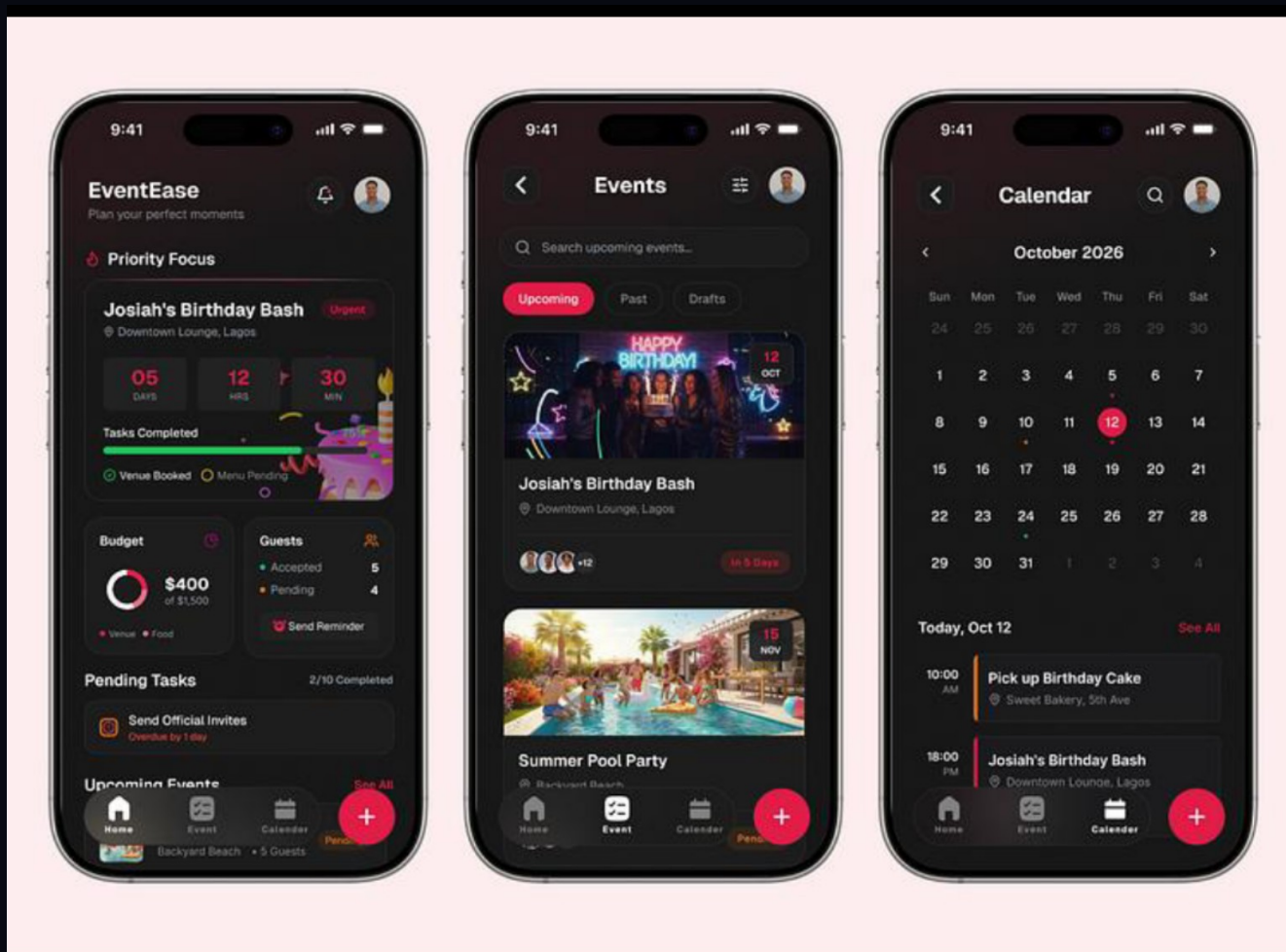
Reference 04 - Event timeline / next-action / event-day scheduling experience



Use for event-day timeline: chronological activities, "Up Next", duration, status and bottom navigation. StarVnt should evolve this into an event-day control layer.

16.5 | Visual Reference 5

Reference 05 - Dark premium event dashboard / events / calendar / priority focus



Use for premium dark UI: priority focus, event cards, budget, guests, pending tasks and calendar. Extract the visual discipline and hierarchy, not the exact design.

17 | Next.js Delivery Checklist

Before calling the customer-facing implementation complete, the team should verify:

- Next.js customer application runs through the approved architecture and environment.
- Authentication and event-level authorization are enforced through the Core Platform.
- Home renders real event data rather than hard-coded mock values.
- Budget and Payment concepts are separated in UI and data access.
- Vendor, task and timeline states are represented clearly.
- Loading / empty / error / permission states exist for major screens.
- Aura+ context is scoped to authorized customer event data.
- Mobile layouts work across real Android/iOS sizes.
- Media is optimized and not embedded as database blobs.
- Analytics / error tracking hooks are in place where approved.
- Critical user actions follow backend business rules and state transitions.
- UI components are reusable and not duplicated across feature screens.
- All major screen/API dependencies are documented.

18 | Final North Star

Product

StarVnt Customer App is not a dashboard for viewing an event. It is a control center for planning, understanding and executing the event.

Customer provides: Event Type + Date + Location + Guest Count + Budget + Preferences



StarVnt understands the event



Recommended Event Plan → Budget Allocation → Requirements → Vendors → Bookings → Tasks → Payments → Timeline → Execution



Aura+ acts as the intelligence layer across the journey.

Reference apps inspire the interface. StarVnt owns the experience.

The objective is not to build more screens. The objective is to build a better customer experience.

— Subho
Founder & CEO
StarVnt Entertainment